

Chapter 4: Object-Oriented Architecture, Dunders & Metaprogramming

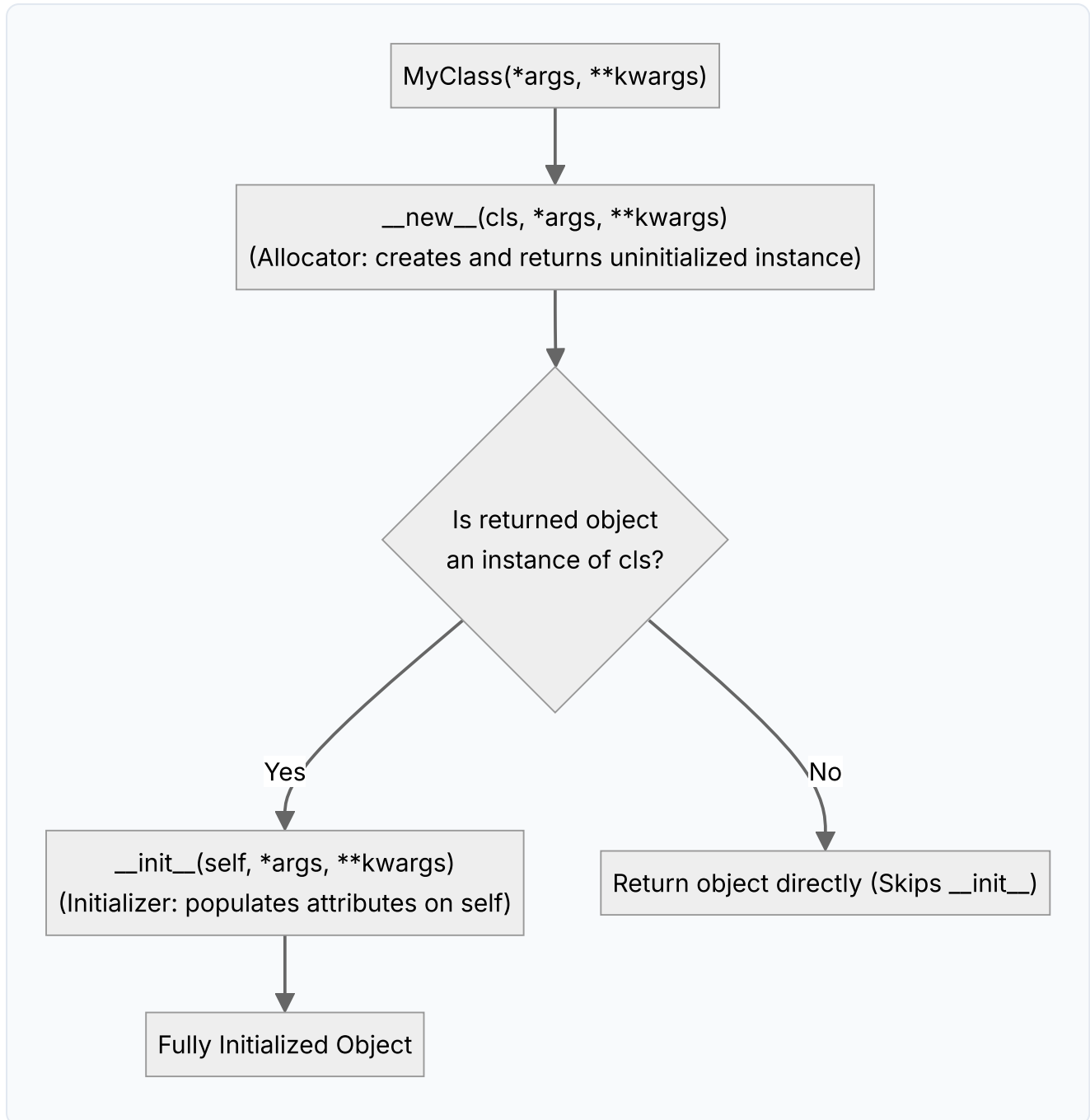
Core Learning Objective: Master Python's object model at the deepest level. Understand instance allocation via `__new__`, C3 Linearization (MRO), the Descriptor Protocol that powers `@property` and ORMs, and how to harness Metaclasses to intercept and transform class definitions.

1. Object Construction: `__new__` vs `__init__`

Zero-Prerequisite Intuition: The "Carpenter vs. Interior Decorator" Metaphor Most developers believe `__init__` creates the object. It does not—it only decorates it!

- **`__new__` is the Carpenter:** Its job is to go into the empty warehouse (heap memory), saw the raw wood, assemble the frame, and physically construct the table (`PyObject`). Without the carpenter, there is literally no physical object in existence.
- **`__init__` is the Interior Decorator:** Once the carpenter finishes and hands over the bare wooden table (`self`), the decorator arrives with paint, tablecloths, and silverware (`self.name = "DiningTable"`). If the decorator never shows up, the table still physically exists—it is just bare and unpainted!

Most developers believe `__init__` is Python's constructor. It is **not**—it is only an initializer.



The CPython Mechanics

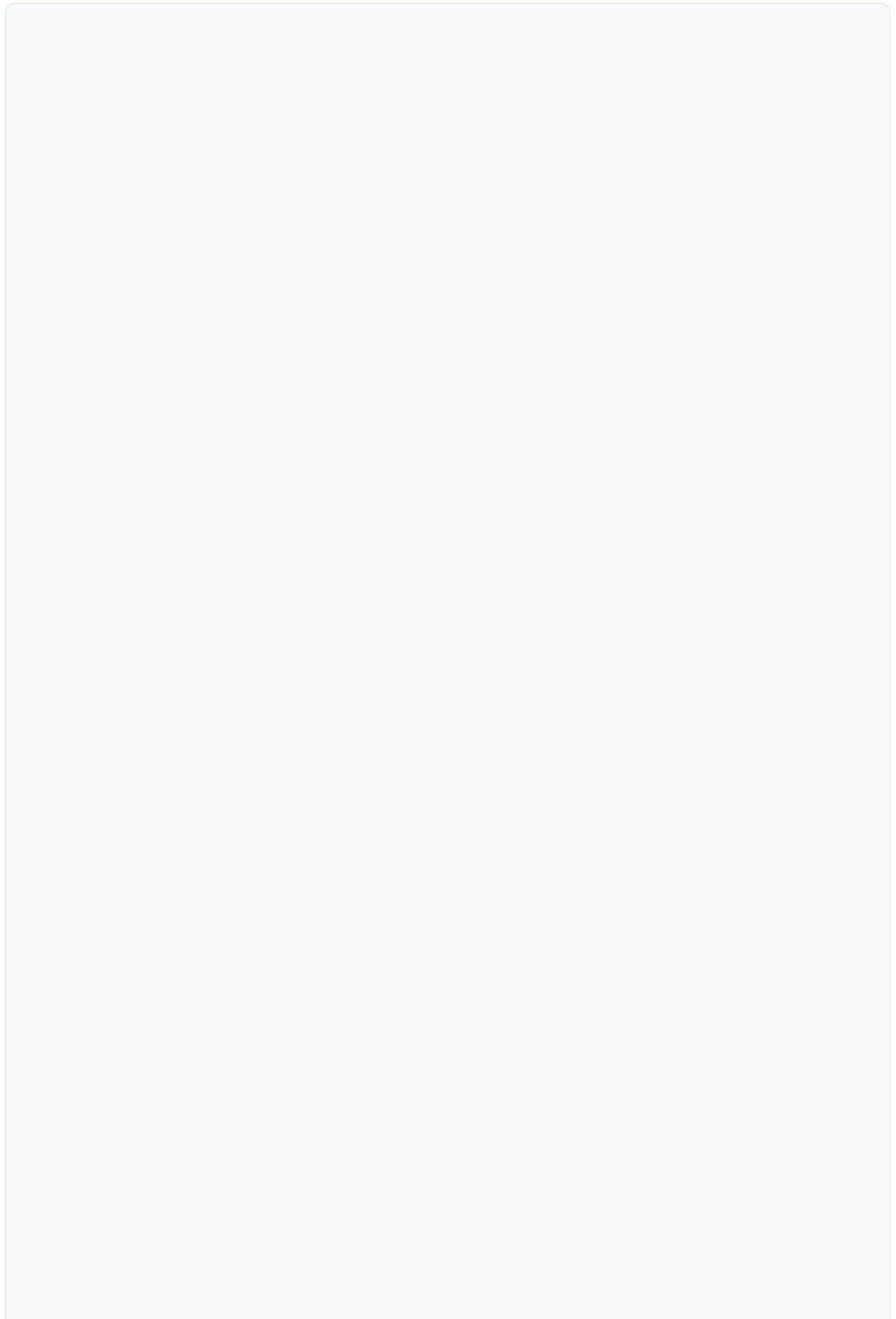
1. `__new__(cls, ...)`: A static method responsible for allocating memory and returning the raw instance. It must return an instance of `cls` for `__init__` to be invoked.
2. `__init__(self, ...)`: Receives the newly created `self` instance to attach fields.

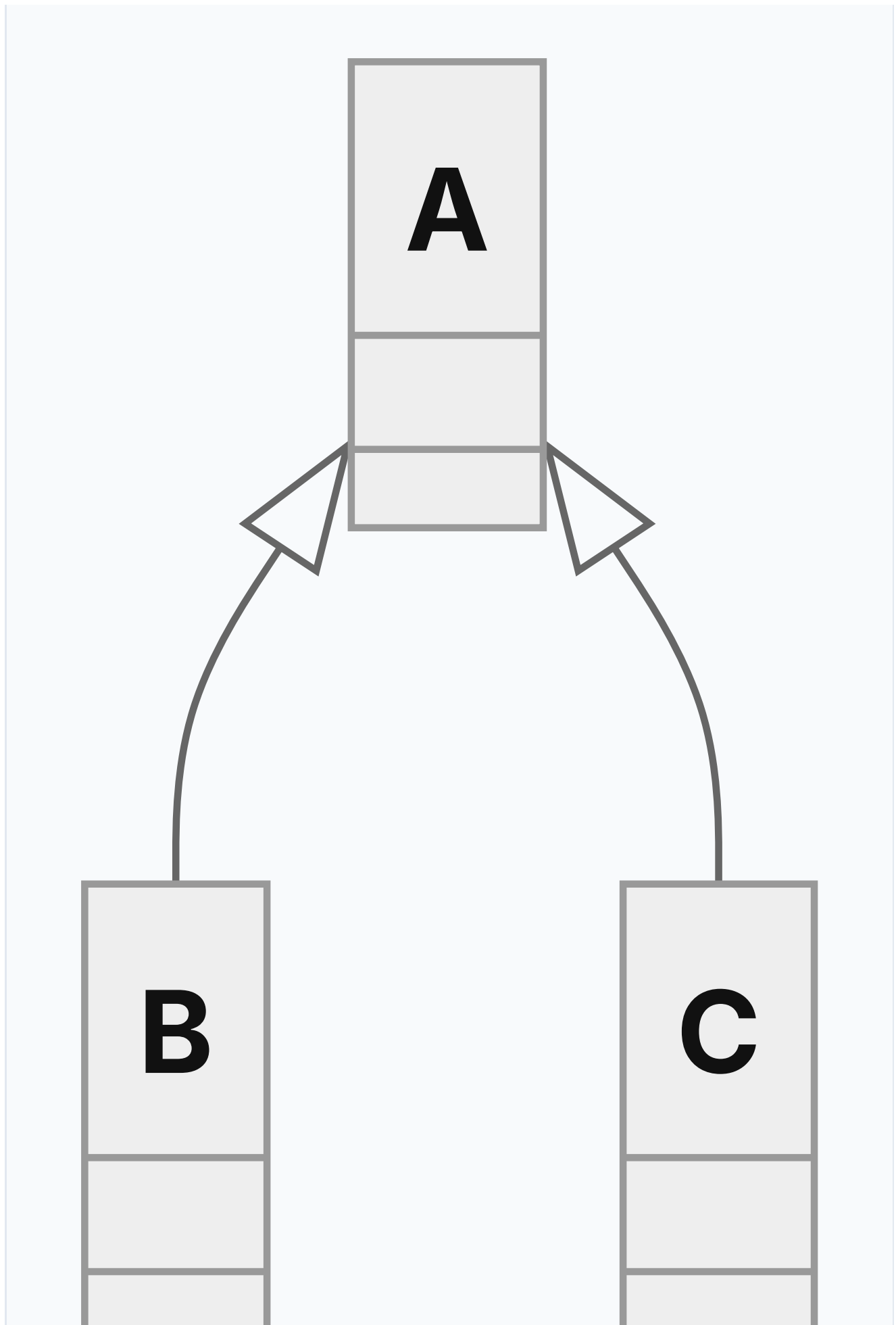
Pattern: Implementing a True Singleton via `__new__`

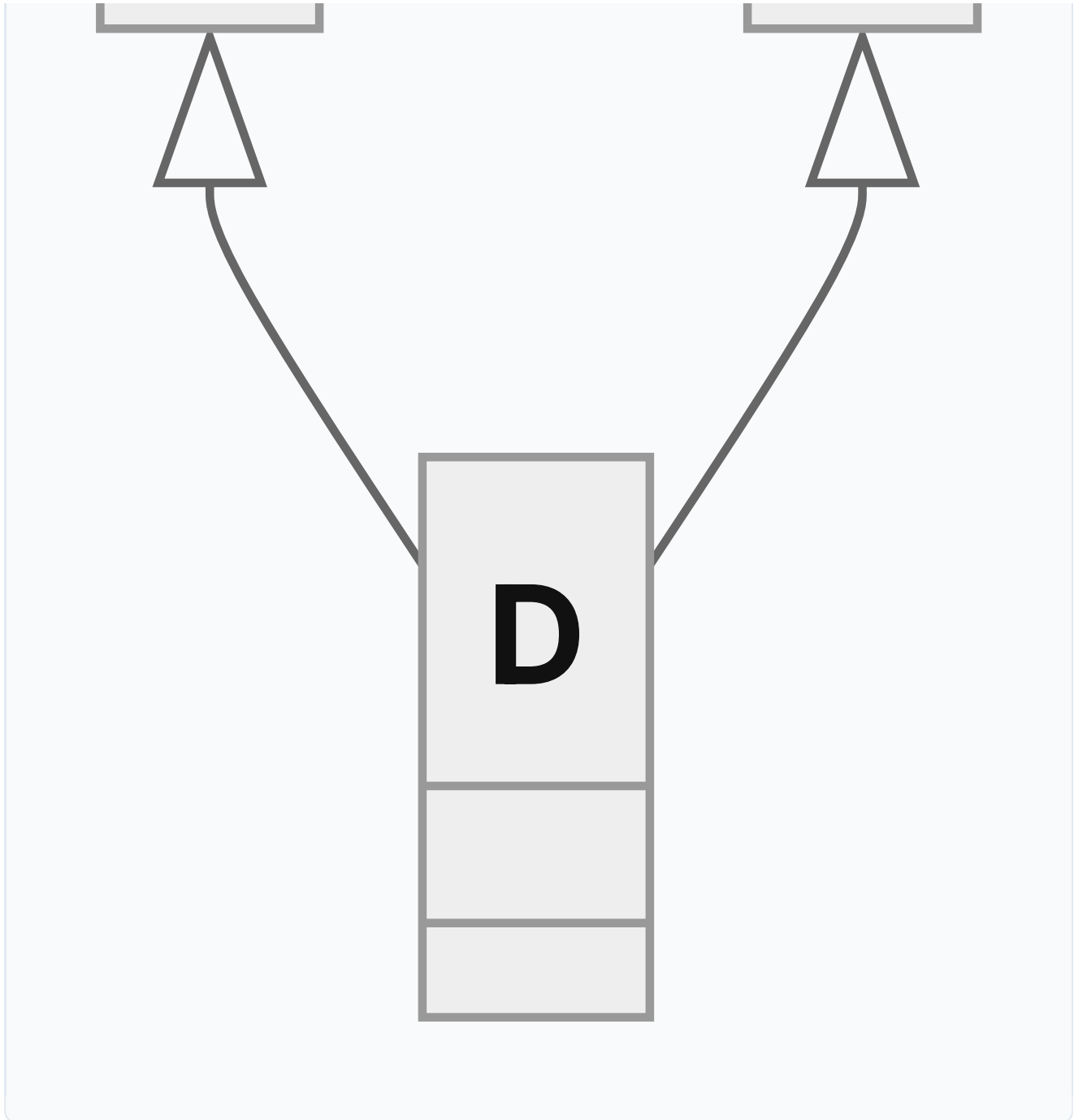


2. Multiple Inheritance & the C3 Linearization Algorithm (MRO)

Python supports multiple inheritance, resolving the classic **Diamond Problem** using the **C3 Linearization Algorithm** to produce a deterministic **Method Resolution Order (MRO)**.







How `super()` Actually Operates

`super()` does **not** simply call the parent class in the source code; it calls the **next class in the runtime caller's MRO chain**.

Output:

IMPORTANT

Notice that `ServiceA.execute()` calls `super().execute()`, which transfers control to `ServiceB`! In isolation, `ServiceA` does not inherit from `ServiceB`. This cooperative call chain works solely because `CompositeService` linearized both into its MRO.

3. Magic (Dunder) Methods: Emulating Core Language Protocols

Dunder ("double underscore") methods allow user-defined classes to hook into Python's native operators and protocols.

1. Attribute Access Hooks: `__getattr__` vs `__getattribute__`

- `__getattribute__(self, name)`: Intercepts **every single attribute lookup**, even if the attribute exists in `self.__dict__`. (Danger: Calling `self.x` inside it causes infinite recursion!).
- `__getattr__(self, name)`: Fallback hook invoked **only when the attribute cannot be found** via normal lookup.

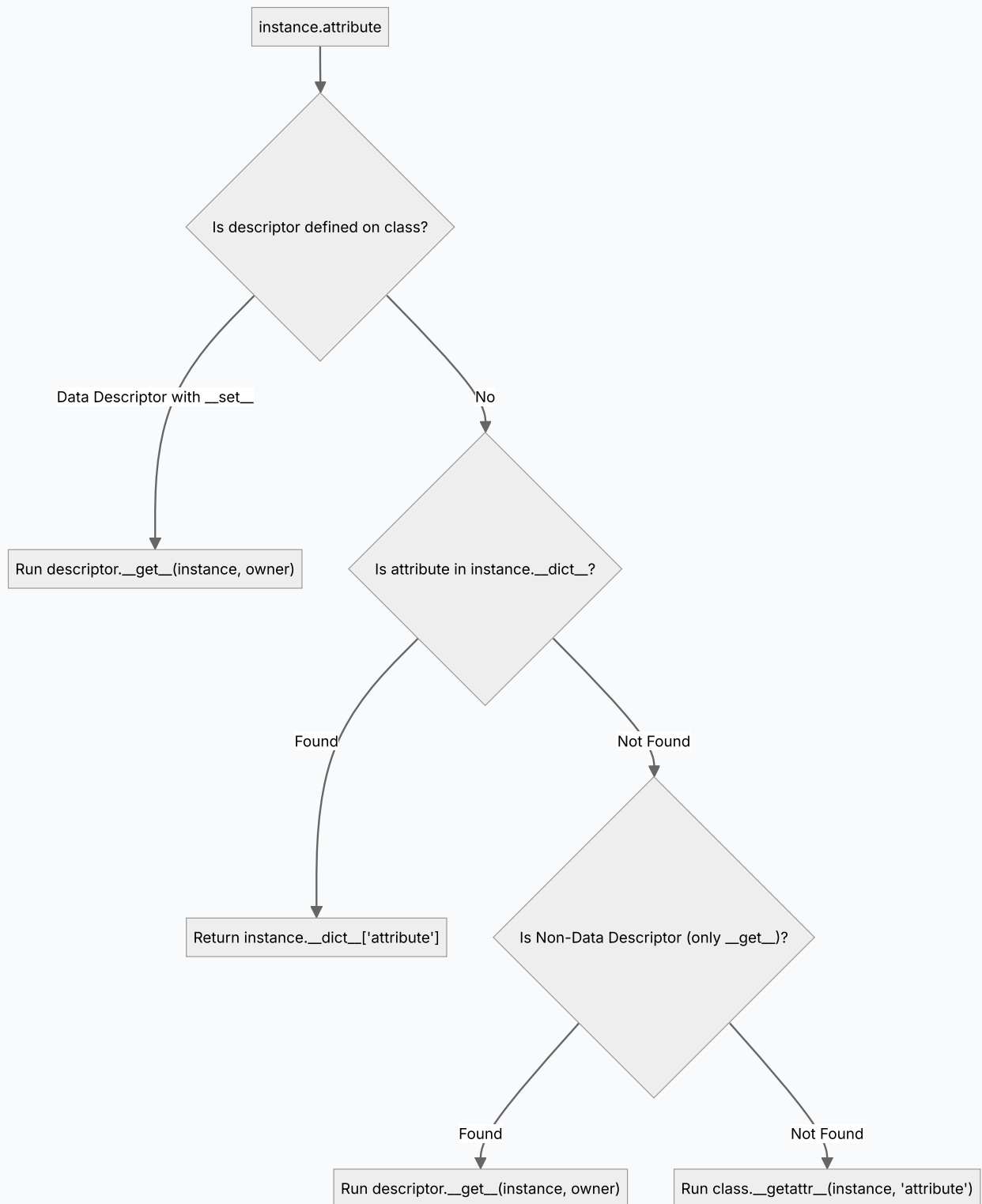
2. Context Manager Protocol: `__enter__` and `__exit__`

4. The Descriptor Protocol: The Engine of Python

Descriptors are the underlying mechanism powering `@property`, `@classmethod`, `@staticmethod`, and every major ORM (SQLAlchemy, Django ORM, Pydantic).

A descriptor is any class that implements at least one of:

- `__get__(self, instance, owner)`
- `__set__(self, instance, value)`
- `__delete__(self, instance)`



Real-World Production Descriptor: Type-Checked ORM Fields

5. Metaclasses: The Classes of Classes

Zero-Prerequisite Intuition: The "Cookie Cutter Factory" Metaphor What is a Metaclass? Think of an individual object (like `emp = Employee("Alice")`) as a **cookie**. The class `Employee` is the **cookie cutter** that shapes the cookie. But where did the cookie cutter come from? Someone had to manufacture that metal cookie cutter in a machine tool shop! A **Metaclass** is the **factory that manufactures cookie cutters**.

- Metaclass = Factory forging the class blueprints (`type`).
- Class = The blueprint / cookie cutter stamping out instances (`Employee`).
- Instance = The cookie (`emp`). By writing a custom Metaclass, you can inspect, modify, or enforce strict rules on the cookie cutter *before* anyone bakes a single cookie (e.g. automatically verifying that every database model has a primary key, or registering API routes dynamically!).

In Python:

- An object is an instance of a **Class**.
- A Class is an instance of a **Metaclass** (`type`).

Dynamic Class Creation with `type()`

The 3-argument form of `type(name, bases, dict)` creates classes at runtime:

Production Metaclass: Auto-Registering API Handlers

Metaclasses allow you to intercept the creation of classes, enforce invariants, or register them in central plugin registries before the class is ever instantiated:

Practice Drills & Interview Verifications

Drill 1: Thread-Safe Singleton Metaclass

Question: Implement a generic `SingletonMeta` metaclass that ensures any class declaring `metaclass=SingletonMeta` can only ever be instantiated once, even across concurrent threads.

Exercises

Three graded exercises for this chapter (two coding, one debugging) with hidden tests:

Attempt first; the reference solutions are in `exercises/_answers/ch04.py`.

Further Reading

- [The Python 2.3 Method Resolution Order \(C3\)](#)
- [Descriptor how-to guide](#)
- [Data model: special method names](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. What is the MRO and which algorithm computes it?
- ▶ 2. How do `__new__` and `__init__` differ?
- ▶ 3. What does `super()` do in multiple inheritance?
- ▶ 4. What is a descriptor?